

1a) Create a registration form using HTML and CSS with the following fields:

- Name
- Email
- Password
- Mobile Number

Apply CSS Flexbox for layout.

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
.container{
```

```
    display:flex;
```

```
    justify-content:center;
```

```
    align-items:center;
```

```
    height:100vh;
```

```
}
```

```
form{
```

```
    display:flex;
```

```
    flex-direction:column;
```

```
    width:300px;
```

```
    gap:10px;
```

```
}
```

```
input{
```

```
    padding:8px;
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div class="container">
```

```

<form>
<h2>Registration Form</h2>

<input type="text" placeholder="Name">
<input type="email" placeholder="Email">
<input type="password" placeholder="Password">
<input type="text" placeholder="Mobile Number">

<button type="submit">Register</button>

</form>

</div>

</body>

</html>

```

Output

```

-----
| Registration Form |
| Name             |
| Email            |
| Password         |
| Mobile Number    |
| [ Register ]     |
-----

```

1b) Validate the registration form using JavaScript:

- Name should not be empty.
- Password length should be at least 6 characters.
- Mobile number should contain exactly 10 digits.

```
<script>
```

```
function validate(){
```

```
    let name=document.getElementById("name").value;
```

```
    let password=document.getElementById("password").value;
```

```
    let mobile=document.getElementById("mobile").value;
```

```
    if(name==""){
```

```
        alert("Name cannot be empty");
```

```
        return false;
```

```
    }
```

```
    if(password.length<6){
```

```
        alert("Password must contain at least 6 characters");
```

```
        return false;
```

```
    }
```

```
    if(!/^d{10}$/.test(mobile)){
```

```
        alert("Mobile number must be 10 digits");
```

```
        return false;
```

```
    }
```

```
    return true;
```

```
}
```

```
</script>
```

Output

Invalid Name:

Alert:
Name cannot be empty

Valid Input:

Alert:
Validation Successful

2a) Design a product catalog page using HTML, CSS Grid, and Flexbox displaying:

- Product Image
- Product Name
- Price

```

<!DOCTYPE html>
<html>
<head>
<style>
.catalog{
display:grid;
grid-template-columns:repeat(3,1fr);
gap:20px;
}

.card{
display:flex;
flex-direction:column;
align-items:center;
border:1px solid black;
padding:10px;
}

img{
width:150px;
height:150px;
}
</style>
</head>
<body>

<div class="catalog">

<div class="card">

<h3>Laptop</h3>
<p>₹50000</p>
</div>

<div class="card">

<h3>Mobile</h3>
<p>₹20000</p>
</div>

</div>

</body>
</html>

```

Output

Laptop	₹50000	
--------	--------	--

Mobile	₹20000	
Headphones	₹1500	

2b) Add an "Add to Cart" button using JavaScript that displays an alert message when clicked.

```
<button onclick="addCart()">Add to Cart</button>
```

```
<script>
function addCart(){
    alert("Product Added to Cart");
}
</script>
```

Output

Alert:

Product Added to Cart

3a) Create a responsive login page using Bootstrap.

Fields:

- Username
- Password

```
<!DOCTYPE html>
<html>
<head>
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>

<body>

<div class="container mt-5">
<div class="row justify-content-center">
<div class="col-md-4">

<form>
<h2>Login</h2>

<input type="text" class="form-control mb-3" placeholder="Username">

<input type="password" class="form-control mb-3" placeholder="Password">

<button class="btn btn-primary w-100">
Login
</button>

</form>

</div>
</div>
</div>

</body>
</html>
```

Output

Login	
Username	
Password	
[Login]	

3b) Perform client-side validation using JavaScript:

- Username should not be empty.
- Password should contain at least 6 characters.

```
<script>
function validate(){
```

```
let user=document.getElementById("user").value;
let pass=document.getElementById("pass").value;

if(user==""){
    alert("Username required");
    return false;
}

if(pass.length<6){
    alert("Password must be at least 6 characters");
    return false;
}

return true;
}
</script>
```

Output

If username empty:

Alert:

Username required

If password < 6:

Alert:

Password must be at least 6 characters

Success:

Alert:

Login Successful

4a) Write an ES6 program using Arrow Functions to calculate:

- Sum of two numbers
- Square of a number

```
const sum=(a,b)=>a+b;
```

```
const square=n=>n*n;
```

```
console.log("Sum =",sum(10,20));
console.log("Square =",square(5));
```

Output:

Sum = 30

Square = 25

4b) Demonstrate Callback Functions by printing:

1. "Data Loading..."
2. "Data Loaded Successfully"

using a callback.

```
function loadData(callback){
    console.log("Data Loading...");
    callback();
}

function success(){
    console.log("Data Loaded Successfully");
}

loadData(success);
```

Output:

```
Data Loading...
Data Loaded Successfully
```

5a) Create a Java application to insert student details into a MySQL database.

Fields:

- Roll No
- Name
- Branch

```
import java.sql.*;

public class InsertStudent {
    public static void main(String args[]) throws Exception {

        Class.forName("com.mysql.cj.jdbc.Driver");

        Connection con=DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/studentdb",
            "root","root");

        PreparedStatement ps=
            con.prepareStatement(
                "insert into student values(?,?,?)");

        ps.setInt(1,101);
        ps.setString(2,"Ravi");
```

```

        ps.setString(3,"CSE");

        ps.executeUpdate();

        System.out.println("Record Inserted");

        con.close();
    }
}

```

Output

Record Inserted

Database Table

RollN	Nam	Branc
o	e	h
101	Ravi	CSE

5b) Write a Java program to display all student records from the database.

```

import java.sql.*;

public class DisplayStudent {
    public static void main(String args[]) throws Exception {

        Connection con=
        DriverManager.getConnection(
        "jdbc:mysql://localhost:3306/studentdb",
        "root","root");

        Statement st=con.createStatement();

        ResultSet rs=
        st.executeQuery("select * from student");

        while(rs.next()){
            System.out.println(
            rs.getInt(1)+" "+
            rs.getString(2)+" "+
            rs.getString(3));
        }
    }
}

```

```
        con.close();
    }
}
```

Output

```
101 Ravi CSE
102 Suresh ECE
103 Kiran IT
```

6a) Create an XML document for a bookstore containing:

- Book ID
- Title
- Author
- Price

```
<?xml version="1.0"?>
```

```
<bookstore>
```

```
<book>
```

```
<bookid>101</bookid>
```

```
<title>Java</title>
```

```
<author>James</author>
```

```
<price>500</price>
```

```
</book>
```

```
</bookstore>
```

Output

XML document created successfully.

```
<bookstore>
```

```
<book>
```

```
<bookid>101</bookid>
```

```
<title>Java</title>
```

```
<author>James</author>
```

```
<price>500</price>
```

```
</book>
</bookstore>
```

6b) Write a DTD to validate the bookstore XML document.

```
<!ELEMENT bookstore (book+)>

<!ELEMENT book
(bookid,title,author,price)>

<!ELEMENT bookid (#PCDATA)>
<!ELEMENT title (#PCDATA)>
<!ELEMENT author (#PCDATA)>
<!ELEMENT price (#PCDATA)>
```

Output

XML Validation Successful (or)
No validation errors found

7a) Write a Java JDBC program to update a student's branch using Roll Number.

```
import java.sql.*;

public class UpdateStudent{
public static void main(String args[]) throws Exception{

Connection con=
DriverManager.getConnection(
"jdbc:mysql://localhost:3306/studentdb",
"root","root");

PreparedStatement ps=
con.prepareStatement(
"update student set branch=? where rollno=?");

ps.setString(1,"ECE");
ps.setInt(2,101);

ps.executeUpdate();

System.out.println("Updated");

con.close();
}
}
```

Output

Updated Successfully

Before Update

RollN o	Nam e	Branc h
101	Ravi	CSE

After Update

RollN o	Nam e	Branc h
101	Ravi	ECE

7b) Write a Java JDBC program to delete a student record using Roll Number.

```
import java.sql.*;

public class DeleteStudent{
    public static void main(String args[]) throws Exception{

        Connection con=
        DriverManager.getConnection(
        "jdbc:mysql://localhost:3306/studentdb",
        "root","root");

        PreparedStatement ps=
        con.prepareStatement(
        "delete from student where rollno=?");

        ps.setInt(1,101);

        ps.executeUpdate();

        System.out.println("Deleted");

        con.close();
    }
}
```

Output

Deleted Successfully

After Deletion

RollNo	Name	Branch
--------	------	--------

No Record
Found

8a) Create a custom HTTP server using Node.js that displays:
"Welcome to Node Server"

```
const http=require('http');

http.createServer((req,res)=>{

res.write("Welcome to Node Server");

res.end();

}).listen(3000);

console.log("Server Running");
```

Browser Output

Open:

http://localhost:3000

Output:

Welcome to Node Server

Terminal

Server Running

8b) Display the current operating system information using the OS module.

```
const os=require('os');

console.log("OS:",os.type());

console.log("Platform:",
os.platform());

console.log("CPU:",
os.cpus().length);
```

Output

```
OS: Windows_NT
Platform: win32
CPU: 8
```

9a) Create a REST API endpoint to update student information using PUT method.

```
const express = require('express');
const app = express();
app.use(express.json());
app.put('/student/:id', (req, res) => {
  res.json({ message: "Student Updated" });
});
app.listen(3000, () => {
  console.log("Server is running on http://localhost:3000");
});
```

Output:

Request

PUT /student/101

Response

```
{
  "message": "Student Updated"
}
Student Updated
```

9b) Create a REST API endpoint to delete a student record using DELETE method.

```
const express = require('express');
const app = express();
app.delete('/student/:id', (req, res) => {
  res.json({ message: "Student Deleted" });
});
app.listen(3000, () => {
  console.log("Server is running on http://localhost:3000");
});
```

Output

Request

DELETE /student/101

Response

```
{  
  "message": "Student Deleted"  
}
```

10a) Create a React Weather component that fetches current weather data using API.

src/WeatherChart.jsx

```
import {useState} from "react";  
function Weather(){  
  const [data,setData]=useState({});  
  const getWeather=async()=>{  
    const res=await fetch(  
      "https://api.openweathermap.org/data/2.5/weather?q=Hyderabad&appid=YOUR_KEY"  
    );  
  
    const result=await res.json();  
  
    setData(result);  
  }  
  
  return(  
    <div>  
  
    <button onClick={getWeather}>  
      Get Weather  
    </button>  
  
    <h3>{data.name}</h3>  
  
    </div>  
  );  
}  
  
export default Weather;
```

src/App.jsx

```
import WeatherChart from "../WeatherChart";  
  
function App() {  
  return (  
    <div>
```

```

        <h2 style={{ textAlign: "center" }}>Weather Data Visualization</h2>

        <WeatherChart />

    </div>

    );}

export default App;

```

Output

After clicking **Get Weather**:

City : Hyderabad

Temperature : 31°C

Humidity : 68%

(Actual values depend on API response.)

10b) Display weather information using a bar chart with Chart.js.

```

import { Bar } from "react-chartjs-2";
import "chart.js/auto"; // This simple line configures everything automatically!

```

```

function WeatherChart() {
  const data = {
    labels: ["Temp", "Humidity"],
    datasets: [
      {
        label: "Weather",
        data:,
        backgroundColor: ["blue", "green"]
      }
    ]
  };

  return <Bar data={data} />;
}

```

```

export default WeatherChart;

```

Install:

```

npm install chart.js react-chartjs-2

```

Output

Bar Chart:

Temperature  30
Humidity  70
Graph displayed visually in browser.

11a) Create a Promise that resolves after 3 seconds and displays:
"Weather Data Retrieved Successfully"

```
const weatherPromise=  
new Promise((resolve)=>{  
  
  setTimeout(()=>{  
  
    resolve(  
      "Weather Data Retrieved Successfully"  
    );  
  
    },3000);  
  
  });  
  
weatherPromise.then(  
  msg=>console.log(msg)  
);
```

Output

(after 3 seconds)

Weather Data Retrieved Successfully

11b) Rewrite the above program using Async/Await.

```
function getWeather(){  
  
  return new Promise((resolve)=>{  
  
    setTimeout(()=>{  
  
      resolve(  
        "Weather Data Retrieved Successfully"      )
```

```
);  
},3000);  
});  
}  
  
async function display(){  
  
  const result=  
  await getWeather();  
  
  console.log(result);  
  
}  
  
display();
```

Output

(after 3 seconds)

Weather Data Retrieved Successfully

12a) Generate a JWT token after successful login using Express.js.

```

const express=require('express');
const jwt=require('jsonwebtoken');

const app=express();

app.use(express.json());

app.post('/login',(req,res)=>{

const token=jwt.sign(
  {user:"admin"},
  "secretkey",
  {expiresIn:"1h"}
);

res.json( {token});

});

app.listen(3000);
Install:
npm install express jsonwebtoken

```

Output:

Request

POST /login

Response

```

{
  "token":
  "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}

```

12b) Verify the JWT token before accessing a protected route.

```

const express = require('express');
const jwt = require('jsonwebtoken');
const app = express();

function verifyToken(req, res, next) {

```

```
// Safely reads the token header, defaulting to an empty string if missing
const token = req.headers.authorization || "";

jwt.verify(token, "secretkey", (err) => {
  if (err) return res.send("Invalid Token");
  next();
});
}

app.get('/profile', verifyToken, (req, res) => {
  res.send("Protected Route Accessed");
});

app.listen(3000);
```

Valid Token Output

Protected Route Accessed

Invalid Token Output

Invalid Token